

School of Computer Science: assessment brief

Module title	Parallel Computation
Module code	COMP3221
Assignment title	Coursework 2
Assignment type and description	MPI Programming Assignment
Rationale	To implement a calculation on a distributed memory system using MPI, including a binary tree using point-to-point communication. To perform timing runs for parallel scaling and to interpret your results.
Guidance	See overpage
Weighting	20%
Submission dead-line	2pm, Thursday 19 th March
Submission method	Gradescope
Feedback provision	Marks and comments returned <i>via</i> Gradescope
Learning outcomes assessed	Apply parallel design paradigms to serial algorithms. Evaluate and select appropriate parallel solutions for real world problems.
Module lead	David Head

1. Assignment guidance

This coursework specification is for school Unix machines only, including the remote access `feng-linux.leeds.ac.uk`. We cannot guarantee it will work on any other environment.

You need to implement an MPI-C program that performs matrix–vector multiplication in parallel. That is, given the $N \times N$ matrix A and the N –vector $\mathbf{x}$ defined on rank 0, you need to calculate $\mathbf{b} = A\mathbf{x}$,

$$b_j = \sum_{i=1}^N A_{ij}x_j \quad ,$$

in parallel, and ensure the full N –vector $\mathbf{b}$ is on rank 0 at the end of the calculation.

In the provided starting code, the matrix A is stored as a one-dimensional array of size N^2 rather than a two-dimensional array of size $N \times N$. This ensures the rows of the matrix are stored together in memory, which makes MPI communication easier. For this flattened array, the matrix element at row `row` and column `col` is indexed as `A[row*N+col]`, and the serial code that performs the multiplication is¹

```
int row, col;
for( row=0; row<N; row++ ) {
    b[row] = 0.0f;
    for( col=0; col<N; col++ )
        b[row] += A[row*N+col] * x[col];
}
```

You will need the material up to and including Lecture 11 to complete this assignment.

2. Assessment tasks

Start by downloading the following files from Minerva:

<code>cwk2.c</code>	: Starting point for your solution.
<code>cwk2_extras.h</code>	: Routines to initialise the problem and check the solution. Do not modify this file or avoid calling its functions.
<code>readme.txt</code>	: Use to provide results of your timing runs, and interpretation (see below).
<code>makefile</code>	: A simple makefile (usage optional).

At the start of the code, the problem size N , the matrix A and vector $\mathbf{x}$ are known only to the process with rank zero. Therefore, a step-by-step solution is

1. Communicate N to all processes. **This (along with tasks 3,4 and 6 below) must be performed using MPI functions.**
2. Each process now allocates the local memory required for their calculations.
3. Distribute the full vector $\mathbf{x}$ to all processes.

¹Indexing starts from 1 in mathematical notation but 0 in C-code.

4. Distribute rows of matrix A among the process so that each rank has the same number of rows (N is known to be divisible by the number of processes at this stage).
5. Perform the local calculations on each rank to calculate equal-sized chunks of the vector $\mathbf{b}$.
6. Collect all of the chunks of $\mathbf{b}$ on rank 0 to give the final answer.

As well as initialising the problem on rank 0, the provided code also checks against the serial calculation and outputs the time taken.

Task 1: Implement the parallel matrix-vector multiplication as efficiently as possible using the material up to and including Lecture 11.

Once you have this working, you should then implement an alternative distribution of N to all processes that uses point-to-point communication **following the second binary tree pattern given in Lecture 11**, except that it will be ‘upside-down.’ This should be automatically called if the number of processes `numProcs` is a power of 2; if not, your code should continue to use your original version. You may find the following useful:

<code>1<<n</code>	: Evaluates as 2^n .
<code>if((n&(n-1))==0)</code>	: Evaluates as <code>true</code> if <code>n</code> is a power of 2.
<code>int lev=1; while(1<<lev<=p)lev++;</code>	: Finds the number of levels <code>lev</code> in a binary tree with <code>p</code> nodes in the first layer.

Task 2: Replace step 1 of task 1 by implementing the second binary tree of Lecture 11 to distribute N to all processes when `numprocs` is a power of 2.

Once you have the final version of your code, perform timing runs for $N = 8192$ to determine the parallel execution time as the number of processes is varied (the provided code already outputs this time). The combination of numbers of processes and nodes is given in the file `readme.txt`, and you should insert your results and discussion into this file, and include it when submitting. Note this includes using multiple machines; see Lecture 8 for how to do this. You should also interpret your results in terms of your understanding of MPI and the architecture you used for these timing runs.

Task 3: Performing timing runs for $N = 8192$ and insert the results into the provided `readme.txt` file, with your interpretation².

3. General guidance and study support

If you have any queries about this coursework, visit the Teams page for this module. If your query is not resolved by previous answers, post a new message. Support will also be available during the timetabled lab sessions.

Note that it is perfectly possible that the best solution to a task is not very sophisticated, given the material covered in lectures. You should identify such cases and implement what may be a simple solution.

²If you get the error ‘There are not enough slots...’, include the `--oversubscribe` option to `mpirun`.

4. Assessment criteria and marking process

Your code will be checked using an autograder on Gradescope to test for functionality. Staff will then inspect your code then allocate the marks as per the mark scheme (see below).

5. Submission requirements

You should upload the files `cwk2.c` and `readme.txt` to Gradescope using the portal for this coursework. This will test your submission for correct functionality, and return immediately with the results. If any of the checks were failed, modify your code and re-submit until all checks are passed. Late penalties will be calculated from the date and time of your final submission.

Note that your submission may appear to work on your test system but fail on the autograder. This means that your solution does not work on all systems and is therefore not robust, which means there was an error in your code (see Lecture 3 for a discussion of non-deterministic behaviour for improper parallel code). **You will lose marks if your submission does not pass all of the tests performed by the autograder**, even if it (fortuitously) appears to work on your system.

If you have added new files, you can also upload them and your modified `makefile`. Ensure all submitted files are in one, flat directory. If doing this it is important to make sure your modified makefile still works on the autograder.

Do not modify `cwk2_extra.h`, or copy any of the content to another file and then modify, as this file will be overwritten with an alternative version for assessment.

6. Academic misconduct and plagiarism

Academic integrity means engaging in good academic practice. This involves essential academic skills, such as keeping track of where you find ideas and information and referencing these accurately in your work.

By submitting this assignment, you are confirming that the work is a true expression of your own work and ideas and that you have given credit to others where their work has contributed to yours.

There is a three-tier traffic light categorisation for using Gen AI in assessments. This assessment is **amber** category: AI tools can be used in an assistive role. Use comments in your code to declare any use of generative AI, making clear what tool was used and to what extent.

Code similarity tools will also be used to check for collusion.

7. Assessment/marking criteria

There are 20 marks in total.

- 5 marks : Correct functionality for a range of values of N up to and including $N = 8192$. Correct calculation of the parallel speed-ups in `readme.txt`.
- 6 marks : MPI implementation of the basic matrix-vector multiplication.
- 6 marks : Binary tree distribution of N when `numprocs` is a power of 2.
- 3 marks : Interpretation of the results of the timing runs.

Note you can still achieve a first-class grade without implementing the binary tree.